Project Aurora-927 — Security & Deployment Assessment

Technical Engineering Report

DRAFT - For engineering review; site verification required

Generated 05 November 2026 (UTC)

Contents

1. Executive Summary and Assessment Scope 3

2. System Context and Component Overview 3

3. Authentication Flow and Keycloak Integration 4

4. JWT Issuance, Validation, and Token Handling 5

5. API Access Controls and Authorization Model 6

6. Database Security and PostgreSQL Protection 6

7. Deployment Architecture and Environment Separation 7

8. Security Risk Assessment and Proposed Mitigations 8

9. Deployment Readiness Checklist 9

10. Final Recommendations and Technical Review Notes 10

1. Executive Summary and Assessment Scope

Project Aurora-927 is a fictional application comprising a React front end, a Django-based backend, Keycloak for identity and access management, and PostgreSQL for persistent data storage. This assessment establishes a security and deployment review framework for the solution, focusing on how these components should be configured, integrated, and operated in a production environment. It is intended to support technical review and deployment planning rather than to report the results of a completed penetration test or code audit.

The assessment considers the principal security boundaries between the browser, Django APIs, Keycloak, PostgreSQL, and the deployment infrastructure supporting them. Particular attention is given to authentication flows, JSON Web Token (JWT) issuance and validation, API authorization, database protection, environment separation, secret management, transport security, logging, and operational readiness. The review also considers failure modes that commonly arise when identity, application, and data services are deployed as a single integrated platform.

Because no implementation repository, configuration export, architecture diagram, infrastructure definition, test evidence, or runtime telemetry was provided, this report does not confirm specific vulnerabilities or claim that controls have been implemented. Any risks identified elsewhere in the report should therefore be interpreted as hypothetical or conditional risks requiring verification against the actual codebase, deployment configuration, and operating procedures. The absence of evidence is not treated as evidence that a control is missing.

The scope includes the React client’s handling of authentication state and access tokens; Django middleware, endpoint protection, and authorization decisions; Keycloak realms, clients, redirect URIs, token settings, and administrative controls; PostgreSQL connectivity, permissions, encryption, backups, and exposure; and the deployment model used to isolate development, staging, and production environments. It also includes security considerations for inter-service communication and the protection of credentials, signing keys, and other sensitive configuration.

The assessment excludes independent validation of business requirements, formal compliance certification, source-code quality beyond security-relevant observations, infrastructure provider controls, and performance or availability testing. Where recommendations depend on application-specific behavior, they should be confirmed through design review, configuration inspection, and controlled testing before deployment.

The intended outcome is a defensible baseline for secure implementation and release approval. Subsequent sections describe the target component model, authentication and token flows, authorization expectations, database safeguards, deployment architecture, prioritized risks and mitigations, and a practical deployment-readiness checklist.

2. System Context and Component Overview

Project Aurora-927 is assessed as a web application composed of a React front end, a Django-based back end, Keycloak for identity and access management, and PostgreSQL for persistent data storage. The system is treated as a representative deployment architecture rather than a verified production environment. Accordingly, descriptions in this section establish the assumed system context for subsequent security analysis; they do not constitute evidence that a particular control, configuration, or vulnerability has been confirmed.

The React application provides the browser-facing user interface and communicates with Django through HTTP-based API requests. In a typical deployment, the front end is delivered as static assets through a web server or content delivery layer, while API requests are routed to Django application services. The browser is expected to obtain authentication tokens through the configured Keycloak integration and present an access token when calling protected API endpoints. Client-side controls may improve user experience, but authorization decisions must remain enforced by the Django service rather than by React code or browser state.

Django serves as the principal application and policy enforcement component. It is assumed to expose application APIs, validate incoming requests, apply authentication and authorization rules, perform input and business-logic validation, and mediate all database access. Depending on the selected implementation, Django may use a standard authentication middleware, a JWT-aware library, or custom token validation logic. The security posture therefore depends on consistent validation of token signatures, issuer, audience, expiry, and relevant claims, together with endpoint-specific permission checks.

Keycloak functions as the external identity provider and authorization server. It is expected to manage users, credentials, clients, realms, roles, groups, and token issuance policies. The React client and Django service have distinct trust responsibilities: the browser-facing client is generally a public client, while server-side credentials, administrative interfaces, and service-to-service integrations require more restrictive handling. Keycloak configuration is therefore part of the application security boundary, even though it is deployed separately from Django.

PostgreSQL provides durable storage for application records and, where configured, operational data associated with users, roles, or audit activities. Django should access the database through a dedicated, least-privileged account over an authenticated and encrypted connection where supported by the deployment environment. Direct database access by the React client or end users is outside the intended trust model. Backup, migration, logging, and administrative access paths also form part of the effective database security boundary.

The deployment context is assumed to include separate configuration for development, testing, staging, and production. Network routing, TLS termination, secrets management, logging, and service discovery may be implemented through infrastructure not specified in the available system description. These unspecified elements should be treated as assessment dependencies and verified during technical review rather than presumed secure.

3. Authentication Flow and Keycloak Integration

Project Aurora-927 should use Keycloak as the central identity provider, while Django remains responsible for protecting application resources and enforcing API-level access decisions. The React client should authenticate through the OAuth 2.0 Authorization Code flow with Proof Key for Code Exchange (PKCE). This approach avoids exposing a client secret in the browser and reduces the risk associated with intercepted authorization codes. The React application should redirect the user to the configured Keycloak realm, where the user authenticates and, if applicable, completes any configured multifactor or identity-provider steps.

After successful authentication, Keycloak returns an authorization code to the registered React redirect URI. The client exchanges this code with Keycloak’s token endpoint using the PKCE verifier. The resulting tokens should be held only for the duration and purpose required by the client. In particular, refresh-token handling should follow the selected Keycloak and client configuration, with secure storage and rotation controls preferred over persistent browser storage. Redirect URIs, allowed web origins, and client settings should be explicitly restricted to approved environments rather than using broad wildcards.

For each protected API request, the React client should send the access token in the HTTP Authorization header using the Bearer scheme. Django should validate the token before processing the request. Validation should include the token signature, issuer, audience where configured, expiration, not-before time, and any required scopes or roles. Django should obtain Keycloak’s signing keys through the issuer’s published discovery and JWKS endpoints, cache them appropriately, and support key rotation without requiring manual redeployment. The application should reject malformed, expired, incorrectly issued, or otherwise unsuitable tokens with an appropriate unauthorized response.

Keycloak configuration should separate realms or clients according to the approved environment model. At minimum, development, testing, and production configurations should not share credentials, redirect URIs, signing assumptions, or administrative access. Client registration should use the least privilege necessary: the

browser-facing client should be public and PKCE-enabled, while confidential clients should be used only for server-side integrations that can protect credentials. Service accounts, administrative endpoints, and direct access to Keycloak management interfaces should be restricted to designated operators and trusted network paths.

The principal implementation dependency is consistent configuration across React, Django, and Keycloak. The issuer URL, client identifier, redirect URIs, audience expectations, clock synchronization, and role or scope mappings must be documented and managed through environment-specific configuration. These points are assessment requirements and design controls; their effectiveness should be confirmed through configuration review and integration testing before deployment. No authentication weakness is asserted here without that verification.

4. JWT Issuance, Validation, and Token Handling

Project Aurora-927 should use Keycloak as the authoritative issuer of JSON Web Tokens (JWTs), with the React client obtaining tokens through an appropriate OpenID Connect flow and presenting access tokens to the Django API. For browser-based applications, Authorization Code Flow with Proof Key for Code Exchange (PKCE) is the preferred design because it avoids exposing a client secret in the frontend and reduces the impact of an intercepted authorization code. The application should request only the scopes and audiences required for the intended API operations.

The Django service must validate every access token independently rather than relying on the frontend, session state, or claims supplied in request headers without cryptographic verification. Validation should include the token signature, issuer (iss), intended audience (aud), expiration (exp), not-before time (nbf) where present, and acceptable signing algorithm. The API should obtain Keycloak's public keys through its OpenID Connect discovery and JWKS endpoints, cache them for operational efficiency, and refresh them when an unfamiliar key identifier (kid) is encountered. Key rotation procedures should be tested to ensure that verification continues without accepting obsolete or untrusted keys.

A valid signature alone does not establish authorization. Django should confirm that the token was issued for the expected realm and API audience, then interpret roles, groups, or scopes according to a documented authorization model. Claims should be mapped conservatively, with explicit handling for missing, duplicated, or unexpected values. The API should reject malformed, expired, incorrectly signed, or incorrectly targeted tokens with an appropriate unauthorized response, while avoiding detailed error messages that disclose validation internals. Clock synchronization across Keycloak, the API, and supporting infrastructure is necessary to prevent avoidable failures at token boundaries.

Access tokens should be short-lived and limited to API access. Refresh tokens, if used, should be handled only through the designated OAuth client flow and should not be sent to the Django API as substitutes for access tokens. The React application should avoid persistent browser storage for sensitive tokens where practical, because local storage and similar mechanisms are accessible to JavaScript executing in the application origin. An in-memory strategy, supported by secure, appropriately configured cookies where applicable, can reduce exposure; the selected approach must also address page refresh, cross-site request forgery, and logout behavior.

Logging must never record complete JWTs, refresh tokens, authorization codes, or sensitive claim values. Security-relevant events such as failed validation, key rotation, unusual issuer or audience values, and repeated authentication failures should be logged with redaction and suitable correlation identifiers. These controls represent recommended design requirements; without implementation evidence or testing results, this assessment does not assert that a specific token-handling vulnerability is present.

5. API Access Controls and Authorization Model

The API should enforce authorization independently of the client interface. React routes and UI controls may improve usability, but they must not be treated as security boundaries. Each protected Django endpoint should validate the authenticated identity and then evaluate whether that identity is permitted to perform the requested operation on the target resource. Requests without a valid access token should receive an authentication failure, while authenticated requests that lack the required permission should receive an authorization failure without revealing unnecessary resource information.

Keycloak should provide the authoritative identity and role context for users and services. A practical model for Project Aurora-927 would combine realm or client roles for broad capabilities with application-level checks for resource ownership and business rules. Examples of coarse-grained roles may include user, operator, and administrator. These roles should be mapped into Django authorization policies rather than scattered across individual views. Where appropriate, API permissions can also be represented through narrowly defined scopes, such as read, create, update, or administrative operations. Roles and scopes should be limited to the minimum access required by each client and workload.

Django REST Framework permission classes, policy objects, or equivalent centralized mechanisms should be applied consistently to every protected endpoint, including less prominent administrative, export, bulk-operation, and background-task interfaces. Authorization should be checked after token validation and before database retrieval or modification. For object-level access, the API should verify that the requested record belongs to the caller’s permitted tenant, project, or ownership boundary. Querysets should be filtered by that boundary where possible, reducing the risk that an identifier-based request can access another user’s data. Direct object references, predictable identifiers, and alternate endpoint paths should not bypass these checks.

Administrative functions require separate treatment. Privileged roles should be granted only to designated operators, and sensitive actions should ideally require stronger assurance, such as Keycloak multi-factor authentication or a controlled administrative network path. The application should avoid relying solely on client-supplied role fields, usernames, tenant identifiers, or ownership attributes. These values must be derived from validated token claims and server-side records, with explicit handling for missing, expired, or inconsistent claims.

Service-to-service access should use dedicated confidential clients and narrowly scoped credentials rather than shared user tokens. Each service should receive only the permissions necessary for its defined API calls, and credential rotation and revocation procedures should be established. Authorization decisions and privileged operations should generate security-relevant audit events containing the actor, action, resource context, outcome, and timestamp, while avoiding sensitive token or personal data. Error responses should remain consistent and should not disclose whether unauthorized resources exist. Authorization policies should be reviewed whenever endpoints, roles, tenancy rules, or business workflows change.

6. Database Security and PostgreSQL Protection

PostgreSQL is a critical trust boundary in Project Aurora-927. The Django application should be the only component permitted to access application data directly, while React clients, Keycloak, reverse proxies, background workers, and operational tooling should interact with the database only through explicitly justified service paths. Database credentials, connection parameters, and cryptographic material must not be embedded in source code, client-side bundles, container images, or configuration committed to version control. They should be supplied through an approved secrets-management mechanism and rotated according to operational policy.

The production database should use a dedicated, non-superuser role for the Django application. This role should receive only the privileges required by the deployed schema and migrations, rather than broad ownership or administrative rights. Migration execution should be separated where practical from routine application access,

using a controlled deployment identity with narrowly defined elevated permissions. PostgreSQL administrative roles must not be used by the application, and access to administrative interfaces should be restricted to approved operators through protected management networks or equivalent controls.

Network exposure should be minimized. PostgreSQL should bind to private interfaces and accept connections only from designated application or worker networks through restrictive firewall and pg_hba.conf rules. Public exposure is not required for normal operation and should be prohibited. Transport encryption should be enabled for database connections, with certificate validation configured where supported by the deployment topology. Connection pooling may improve resilience and resource use, but pool limits, idle timeouts, and failure behavior should be defined to prevent exhaustion during traffic spikes or dependency outages.

Django’s database configuration should enforce secure defaults, including parameterized queries through the framework’s data-access mechanisms and appropriate transaction handling. Raw SQL, if required, should be reviewed for injection risk, privilege assumptions, and unintended data disclosure. Schema migrations should be version-controlled, reviewed, and applied through an auditable release process. Destructive changes require particular care because rollback may be incomplete without a verified backup and recovery procedure.

Sensitive data stored in PostgreSQL should be identified and classified before deployment. Application-level protections may be necessary for credentials, tokens, personal information, or other confidential values; hashing should be used for passwords, while encryption may be appropriate for data that must later be recovered. Database logs, audit records, backups, and exports can reproduce sensitive data and therefore require equivalent access controls, retention rules, and protection.

Backups should be encrypted, access-controlled, and tested through documented restoration exercises. Availability planning should address backup frequency, point-in-time recovery requirements, storage durability, and recovery ownership. Monitoring should alert on authentication failures, unexpected connection sources, privilege changes, schema alterations, storage pressure, replication or backup failures, and abnormal query behavior. These controls represent required design protections; no database vulnerability or recovery performance is confirmed without environment-specific configuration review and testing.

7. Deployment Architecture and Environment Separation

Project Aurora-927 should be deployed as a set of isolated services rather than as a single host process. A typical production arrangement places the React application behind a reverse proxy or content delivery layer, with HTTPS termination, security headers, and controlled routing to the Django API. The Django application should run in one or more private application subnets or protected container workloads. Keycloak and PostgreSQL should not be directly reachable from the public internet; access should be limited to the components and administrative networks that require it.

The reverse proxy should expose only the required web and API endpoints. Administrative interfaces for Django, Keycloak, database management, and infrastructure tooling should use separate access paths protected by network controls and strong operator authentication. Where a load balancer is used, it should forward only approved headers and preserve secure protocol information correctly so that Django generates secure URLs and applies appropriate CSRF and cookie behavior. TLS certificates, private keys, and application secrets should be stored in an approved secrets-management mechanism rather than embedded in source code, container images, or deployment manifests.

Environment separation should be explicit across development, testing, staging, and production. Each environment should have independent configuration, credentials, signing keys, databases, and Keycloak realms or equivalent tenant boundaries. Production tokens and user data must not be reused in lower environments. Test data should be synthetic or sanitized, and any data export process should document authorization, minimization, retention, and secure deletion requirements. Network rules should prevent lower-trust environments from initiating connections to

production services.

Containerized deployment can improve repeatability, but images should be built from controlled base images, scanned through the delivery pipeline, and promoted using immutable version identifiers. Runtime containers should use non-root accounts where feasible, read-only filesystems where compatible, restricted Linux capabilities, resource limits, and minimal mounted volumes. Kubernetes or another orchestration platform should apply namespace, service-account, ingress, and network-policy boundaries. If virtual machines are used instead, equivalent segmentation and least-privilege controls should be implemented at the host and firewall layers.

Deployment pipelines should separate build, approval, and release responsibilities. Changes should be reviewed, dependencies locked and monitored, and production releases performed through an auditable process. Configuration differences between environments should be intentional and documented, not maintained through ad hoc manual changes. Database migrations should be versioned and executed by a controlled deployment identity with narrowly scoped permissions.

The architecture described above is a recommended target state, not evidence that these controls currently exist. Confirmation requires review of deployment manifests, network diagrams, identity and secrets configuration, pipeline definitions, and runtime settings. Any missing isolation, publicly exposed administrative service, shared credential, or uncontrolled production access should be treated as a deployment risk requiring remediation before release.

8. Security Risk Assessment and Proposed Mitigations

The following assessment identifies plausible risks for the hypothetical Aurora-927 architecture based on its use of React, Django, Keycloak, JWTs, and PostgreSQL. These items are risk conditions requiring verification, not confirmed vulnerabilities or statements of observed compromise. Final severity should be validated through configuration review, source inspection, and controlled security testing.

1. Improper JWT validation — High. APIs may accept tokens without fully validating the signature, issuer, audience, expiration, or permitted algorithm. Enforce strict JWT validation using trusted Keycloak metadata, require expected claims, reject expired or malformed tokens, and prevent algorithm substitution. Centralize validation behavior and log authentication failures without exposing token contents.

2. Excessive token lifetime or unsafe browser storage — High. Long-lived access tokens or storage in browser-accessible locations could increase the impact of token theft. Use short access-token lifetimes, carefully managed refresh-token rotation, secure cookie settings where compatible with the application, and a documented logout and revocation strategy. Never place tokens in URLs or application logs.

3. Inadequate API authorization — High. Successful authentication may be incorrectly treated as sufficient access, allowing users to view or modify resources belonging to other users or roles. Apply server-side, object-level and action-level authorization on every protected endpoint. Define role and scope mappings explicitly, deny by default, and test horizontal and vertical privilege boundaries.

4. Keycloak misconfiguration — High. Incorrect client settings, redirect URIs, realm policies, or administrative permissions could enable account takeover or unauthorized token issuance. Restrict redirect URIs to approved HTTPS origins, separate administrative clients from application clients, enforce strong administrator authentication, and review realm events and configuration changes.

5. Injection and unsafe data handling — High. Django or database interactions could become vulnerable if raw SQL, unsafe deserialization, or insufficient input validation is introduced. Prefer Django’s parameterized ORM, validate input at trust boundaries, encode output appropriately, and apply narrowly scoped database permissions. Security review should specifically examine file uploads, search parameters, and administrative functions.

6. PostgreSQL exposure or credential compromise — High. Public database access, weak credentials, or excessive service permissions could expose application data. Place PostgreSQL on private network segments, restrict inbound access, use managed secrets or equivalent protected storage, encrypt connections, rotate credentials, and grant the application only required privileges.

7. Cross-site scripting and cross-site request forgery — Medium. React rendering of untrusted content or unsafe state-changing requests could permit account actions through a victim’s browser. Avoid unsafe HTML rendering, sanitize approved rich content, configure an effective content security policy, use secure cookie attributes, and apply CSRF defenses where cookie-based authentication is used.

8. Deployment and observability weaknesses — Medium. Debug settings, exposed management endpoints, missing security headers, or logs containing credentials and personal data could increase attack impact. Separate development, staging, and production configuration; disable debug mode in production; restrict administrative interfaces; patch dependencies; and implement centralized, access-controlled logging with retention and alerting.

Residual risk should be accepted only after mitigation owners, verification activities, and review dates are recorded. Any confirmed vulnerability must be tracked separately from this hypothetical risk register and prioritized according to exploitability, affected assets, and business impact.

9. Deployment Readiness Checklist

The following checklist should be completed before promoting Project Aurora-927 to production. Each item should be supported by configuration evidence, review records, or an explicit risk acceptance decision. A checklist item should not be marked complete solely because the relevant feature exists; its secure configuration, operational ownership, and behavior under failure conditions should also be verified.

Application and API readiness should include confirmation that production settings disable Django debug mode, restrict allowed hosts, enforce secure cookie attributes, and configure trusted origins and proxy behavior correctly. CORS should permit only approved front-end origins, and unnecessary administrative, diagnostic, or development endpoints should be removed or restricted. API authorization must be reviewed for every protected route, including object-level access checks where users can reference resources by identifier. Error responses and logs should be checked to ensure they do not expose credentials, tokens, stack traces, database details, or other sensitive information.

Keycloak readiness should include verification of the production realm configuration, client identifiers, redirect URIs, permitted origins, enabled flows, and administrative access controls. Client secrets and signing-related configuration must be stored outside source control and supplied through an approved secrets-management process. Token validation in Django should use the intended issuer, audience, signature algorithm, and key-discovery configuration, with appropriate handling for expired, malformed, revoked, or incorrectly scoped tokens. Clock synchronization between relevant systems should be confirmed because token time claims depend on consistent system time.

Database and data-protection checks should confirm that PostgreSQL is reachable only from required application or administrative networks. Production credentials should be unique, least-privileged, rotated according to organizational policy, and unavailable to front-end code. TLS requirements, connection settings, backup encryption, backup access controls, and restoration procedures should be reviewed. Migrations should be tested through the intended deployment process, with a rollback or recovery approach defined for failed schema changes. Sensitive data retention and deletion requirements should be documented and implemented.

Infrastructure readiness should include review of network boundaries, firewall or security-group rules, ingress configuration, TLS certificate management, security headers, and container or host hardening. Images and dependencies should be built from controlled sources, scanned through the approved process, and run without unnecessary privileges. Environment-specific configuration must be separated so that development credentials,

test realms, debug settings, and non-production data cannot enter production.

Operational readiness requires centralized logging, alerting, health monitoring, and documented ownership for authentication failures, authorization anomalies, database connectivity issues, and certificate or key changes. Secrets rotation, incident response, vulnerability remediation, and emergency access procedures should be tested or reviewed. A final go-live decision should record unresolved risks, compensating controls, responsible owners, and target dates. Production deployment should proceed only after critical security dependencies are complete or formally accepted by the designated risk authority.

10. Final Recommendations and Technical Review Notes

Project Aurora-927 should proceed toward deployment only after the implementation team has verified the security controls described in this assessment against the actual application configuration, infrastructure, and operational procedures. The risks identified in this report are assessment considerations and potential weaknesses, not confirmed vulnerabilities, unless independently demonstrated through code review, configuration review, or authorized testing. Final approval should therefore be based on documented evidence rather than assumptions about default framework behavior.

The highest priorities are to enforce consistent JWT validation across all protected Django API endpoints, restrict authorization decisions to explicit roles or permissions, and ensure that Keycloak configuration is separated appropriately between environments. Token signature, issuer, audience, expiration, and relevant claims should be validated according to the deployment design. APIs should fail closed when tokens are missing, malformed, expired, or issued by an untrusted authority. Administrative functions should receive separate review because an authentication failure or excessive privilege in these paths could have disproportionate impact.

Database protection should be treated as a layered responsibility. PostgreSQL should not be directly exposed to untrusted networks, and application credentials should be unique, minimally privileged, and managed through an approved secret-management process. Backups, logs, and temporary exports may contain sensitive information and should receive controls equivalent to the primary database. Where feasible, encryption in transit, restricted administrative access, connection limits, and auditable database activity should be enabled and documented.

Deployment readiness should include a repeatable process for building, configuring, updating, and rolling back each component. Production settings must not inherit development secrets, permissive cross-origin rules, debug behavior, or broad network access. Keycloak, Django, PostgreSQL, and the React frontend should have defined ownership, supported versions, vulnerability-management procedures, and recovery responsibilities. Security configuration should be represented as controlled code or documented change procedures to reduce environment drift.

Before release, the team should complete an authorized technical review covering representative authentication flows, token rejection cases, role boundaries, error handling, secret exposure, dependency status, database access paths, logging, and backup restoration procedures. Results should identify the test scope, date, environment, limitations, and unresolved findings; no security property should be recorded as verified solely because it is expected from a framework or product default.

Residual risks should be accepted formally by an accountable owner, with compensating controls and review dates where remediation is deferred. After deployment, monitoring should cover authentication failures, unusual authorization patterns, administrative activity, service health, and configuration changes without logging tokens, passwords, or unnecessary personal data. A scheduled reassessment is recommended after material architecture changes, identity-provider changes, major dependency upgrades, or significant changes to data sensitivity.